

Vision Day2 HW1 레포트

로봇 예비단원 주호진

1. 과제

HSV 색상 기반 공 검출 및 단안거리추정

HSV 기반 노란색 공 검출 → 공 영역 추출 → 단안 카메라 기반 거리 추정 파이프라인 구현

아서 설명한 두가지의 거리 추종 로직 구현

각 방법으로 산출된 거리 추정 결과를 실제 거리와 비교하여 오차를 분석하고, 거리 추종 성능을 평가하여 보고서에 작성

2. 구현

기본 세팅

$$Z = \frac{f_x \cdot W}{w} \quad D = \frac{h}{\tan(\theta + \arctan y_n)} \quad , \quad y_n = (v - c_y) / f_y$$

실측이 필요한 값: W(공의 지름), h(방법 B에서 카메라의 높이)



W = 자로 측정했을 때 약 5.5cm

h = 카메라 자체의 높이 약 5cm, 아래에 설치한 박스 가로 17.5cm, 10cm로 측정하고 진행

헤더파일

```

rclcpp::Subscription<img>::SharedPtr image_sub;
rclcpp::Subscription<camInfo>::SharedPtr info_sub;
rclcpp::Subscription<tiltStatus>::SharedPtr tilt_sub;
void imageCallback(const img::SharedPtr msg);
void cameraInfoCallback(const camInfo::SharedPtr msg);
void tiltStatusCallback(const tiltStatus::SharedPtr msg);

```

Insta360에서 보내주는 image, fx, fy, cy값을 받아올 camera info, 각도 값을 받아올 tilt 구독을 선언해준다.

```
double DistanceComputationA();
```

```
int width;
```

```

double fx = 0.0; // /camera1/compressed_info의 k[0]
double real_width = 0.055; // 1m = 1, 5.5cm = 0.055
double correctionA = 0.50 / 0.29;

```

A방법에서 사용하는 변수를 선언하였다.

Compressed_info topic에서 k의 [0], [4], [5] 인덱스가 각각 fx, fy, cy 값이다.

header:

...

k:

- 471.953641

- 0.0

- 309.509126

- 0.0

- 476.574144

- 228.222101

- 0.0

- 0.0

- 1.0

...

방법 A에서는 50cm 부근에서 29cm라고 나와서 리턴에 0.5 / 0.29를 곱하여 보정하였다.

```
double DistanceComputationB();
```

```
int height;
```

```
double fy = 0.0;           // /camera1/compressed_info의 k[4]
double cy = 0.0;           // /camera1/compressed_info의 k[5]
double v = 0.0;            // 물체의 y축 바닥 좌표
double theta = 0.0;        // tilt 각도
double h = 0.175 + 0.05;   // 가로 10cm, 세로 17.5cm, 카메라 5cm
double correctionB = 1.00 / 0.12;
```

A방법에서 사용하는 변수를 선언하였다. fy와 cy는 Compressed_info topic에서 받아오고 theta는 Pan_tilt_status topic의 tilt값(1/1000도)을 받아오도록 했다.

Pan_tilt_status topic

header:

...

tilt_range:

- -324000

- 360000

steps:

- 3600

- 3600

방법 B에서는 오차가 커서 1m 부근에서 나온 0.12m 값을 역으로 계산해서 리턴에 곱해 보정해 주었다.

소스파일

```

void Hw1Node::cameraInfoCallback(const camInfo::SharedPtr msg)
{
    fx = msg->k[0];
    fy = msg->k[4];
    cy = msg->k[5];
}

void Hw1Node::tiltStatusCallback(const tiltStatus::SharedPtr msg)
{
    double tilt_degree = msg->tilt / -1000.0;

    theta = tilt_degree * M_PI / 180.0; // rad
}

```

각 topic에서 필요한 값이 업데이트 되도록 콜백 설정하였다.

```

void Hw1Node::imageCallback(const img::SharedPtr msg)
{

```

```

    cv_bridge::CvImagePtr cv_ptr;
    cv_ptr = cv_bridge::toCvCopy(msg, "bgr8");

    cv::Mat image = cv_ptr->image;

    if (image.empty())
        std::cout << "경로 확인";

    // 블러링
    cv::medianBlur(image, image, 5);

    cv::Mat hsv;
    cv::cvtColor(image, hsv, cv::COLOR_BGR2HSV);
    // H 0 ~ 179, S 0 ~ 255, V 0 ~ 255
    // H 범위로 목표 색상을 선택, S 하한으로 저채도 영역을 제외, V 하한으로 저명도 영역을 제외

    cv::Mat mask_yellow;
    cv::inRange(hsv, cv::Scalar(20, 100, 100), cv::Scalar(35, 255, 255), mask_yellow);

    // Clsoing
    cv::Mat kernel = cv::getStructuringElement(cv::MORPH_ELLIPSE, cv::Size(3, 3));
    cv::morphologyEx(mask_yellow, mask_yellow, cv::MORPH_CLOSE, kernel);

    std::vector<std::vector<cv::Point>> contours;
    cv::findContours(mask_yellow, contours, cv::RETR_EXTERNAL, cv::CHAIN_APPROX_SIMPLE);
    for (const auto &contour : contours)

```

Vision 1일차에서 hsv를 이용해 이미지 바운딩 박스를 추출하는 코드를 가져와서 노란색을 추출하도록 했다.

```

for (const auto &contour : contours)
{
    double area = cv::contourArea(contour);
    if (area > 30) // 작은 점 인식 방지
    {
        cv::Rect box = cv::boundingRect(contour);

        width = box.width;
        height = box.height;

        v = (double)(box.y + box.height); // box.y: 좌측 상단 꼭짓점 y

        double distanceA = DistanceComputationA();
        double distanceB = DistanceComputationB();

        cv::rectangle(image, box, cv::Scalar(0, 255, 255), 5);

        std::string text = "DistA: " + std::to_string(distanceA).substr(0, 4) + "m" +
                           " | DistB: " + std::to_string(distanceB).substr(0, 4) + "m";

        cv::putText(image, text, cv::Point(box.x, std::max(20, box.y - 10)), cv::FONT_HERSHEY_SIMPLEX, 0.6,
                    cv::Scalar(0, 255, 255), 2);

        break;
    }
}

cv::imshow("image", image);
cv::waitKey(1); // 동영상

```

점 등 작은 박스가 인식되는 경우를 무시하기 위해 일정 크기 이상인 박스만 인식되도록 했다.

박스를 인식하면 해당 박스의 가로, 세로, v = 박스 아랫점(y 좌표)를 구하고 각 방법으로 거리를 구하도록 했다.

```

//  $Z = f_x \cdot W / w$  ( $W$  = 실제 크기,  $w$  = 화면 크기)
double Hw1Node::DistanceComputationA()
{
    if (fx == 0.0 || width <= 0)
        return 0.0;

    return correctionA * (fx * real_width) / (double)width;
}

```

```

double Hw1Node::DistanceComputationB()
{
    if (fy == 0.0)
        return 0.0;

    //  $yn = (v - cy) / fy$ ,  $\alpha = \arctan(yn)$ 
    double yn = (v - cy) / fy;
    double alpha = std::atan(yn);

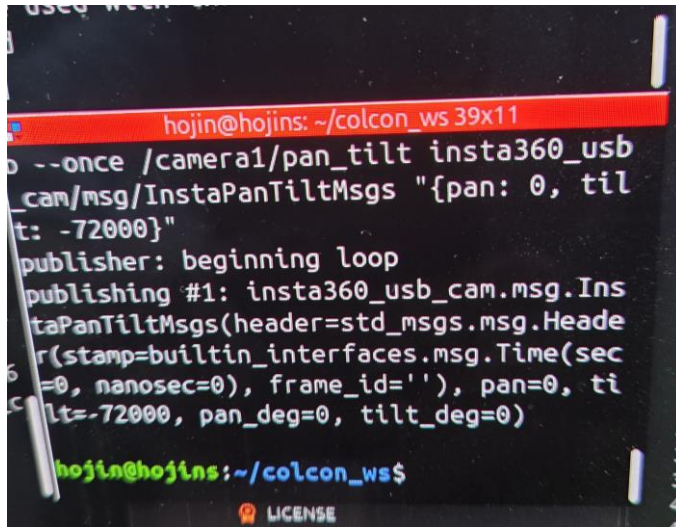
    if (std::abs(std::tan(theta + alpha)) < 1e-6)
        return 0.0; // 분모 != 0이므로 예외처리

    //  $D = h / \tan(\theta + \alpha)$ 
    return correctionB * h / std::tan(theta + alpha);
}

```

각 공식에 맞는 값을 대입하여 return하도록 했다.

3. 결과 사진

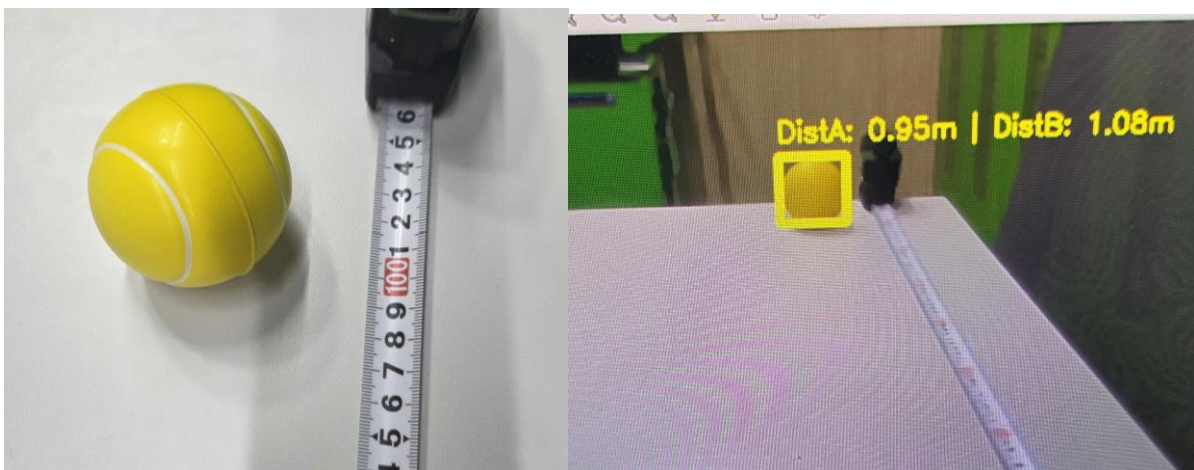
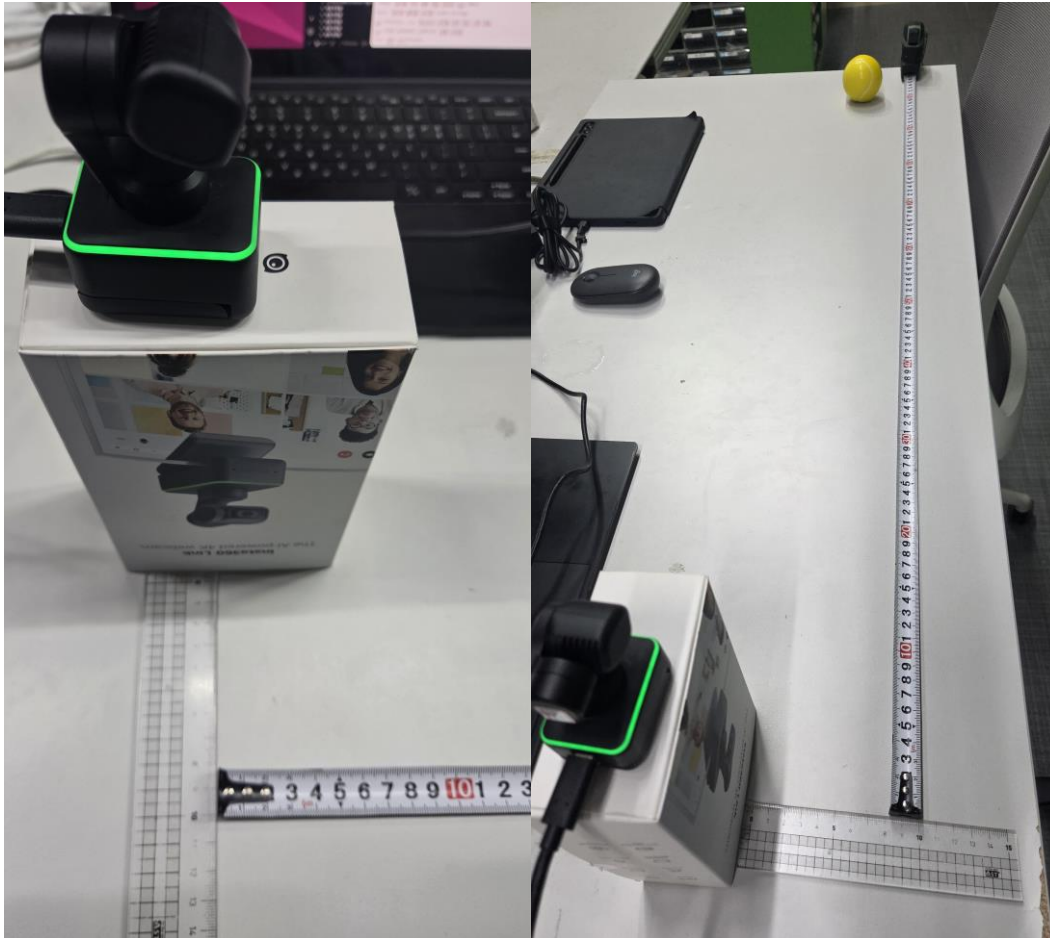
A terminal window with a black background and red title bar. The title bar text is 'hojin@hojins: ~/colcon_ws 39x11'. The terminal shows a ROS2 command being executed: 'ros2 topic pub --once /camera1/pan_tilt insta360_usb_cam/msg/InstaPanTiltMsgs "{pan: 0, tilt: -72000}"'. The output shows a publisher starting a loop and publishing a message of type 'insta360_usb_cam.msg.InstapantiltMsgs'. The message content is displayed as 'r(stamp=builtin_interfaces.msg.Time(sec=0, nanosec=0), frame_id=''), pan=0, tilt=-72000, pan_deg=0, tilt_deg=0)'. The prompt 'hojin@hojins:~/colcon_ws\$' is visible at the bottom.

```
hojin@hojins: ~/colcon_ws 39x11
ros2 topic pub --once /camera1/pan_tilt insta360_usb
_cam/msg/InstaPanTiltMsgs "{pan: 0, til
t: -72000}"
publisher: beginning loop
publishing #1: insta360_usb_cam.msg.Inst
taPanTiltMsgs(header=std_msgs.msg.Heade
r(stamp=builtin_interfaces.msg.Time(sec
=0, nanosec=0), frame_id=''), pan=0, ti
lt=-72000, pan_deg=0, tilt_deg=0)
hojin@hojins:~/colcon_ws$
```

ros2 topic pub --once /camera1/pan_tilt insta360_usb_cam/msg/InstaPanTiltMsgs "{pan: 0, tilt: -72000}"

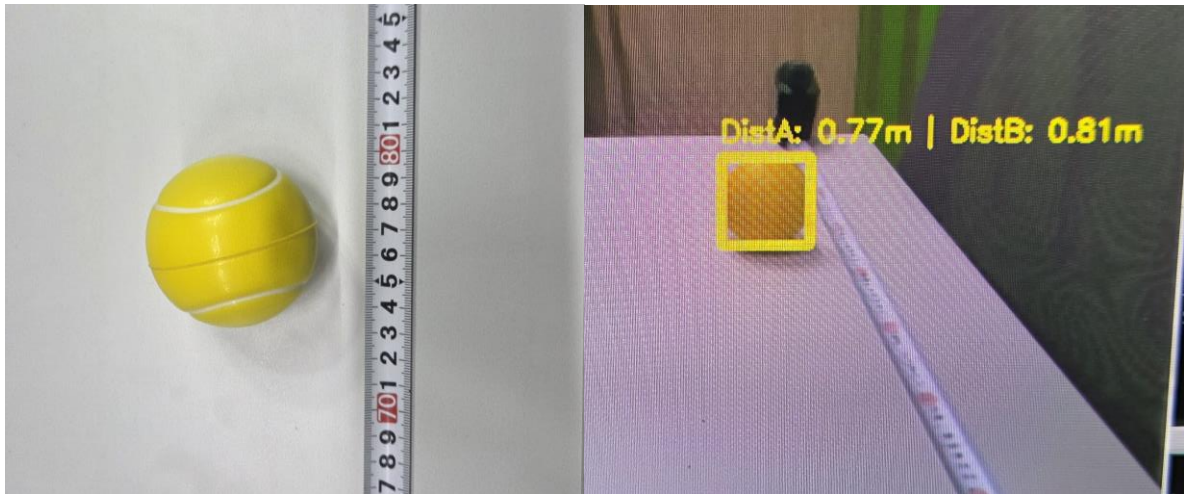
각도: -72도로 설정

박스는 세로로 세운 다음 아래 사진과 같이 세팅하였다.



실제 거리: 1m | 방법 A: 0.95m | 방법 B: 1.08m

오차 | A: -0.05 | B: +0.08



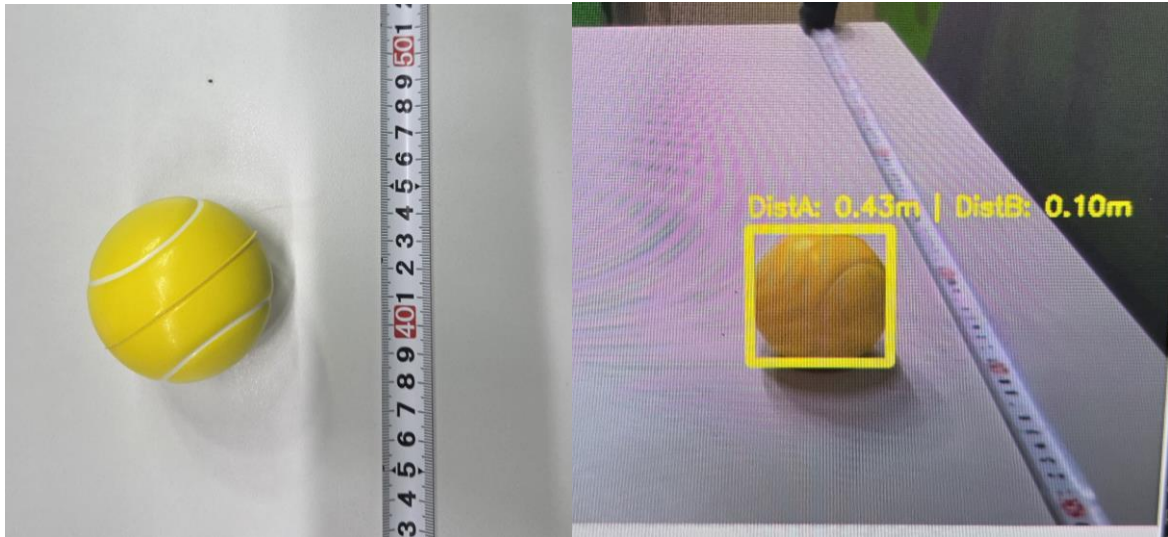
실제 거리: 0.75m | 방법 A: 0.77m | 방법 B: 0.81m

오차 | A: +0.02 | B: +0.06



실제 거리: 0.50m | 방법 A: 0.54m | 방법 B: 0.40m

오차 | A: +0.04 | B: -0.10



실제 거리: 0.40m | 방법 A: 0.43m | 방법 B: 0.10m

오차 | A: +0.03 | B: -0.30

일단 테스트했던 1m정도의 거리에서는 A방법이 B방법보다 매우 우수하게 측정되었다.

위 테스트 환경에서 A의 경우 납득할 정도의 오차가 측정되었는데 B방법의 경우 계수를 보정했던 1m에서 벗어날수록 오차가 점점 커져 40cm 부근에서 측정했을 때는 사용할 수 없는 값이 나왔다.

방법 A는 비교적 간단한 선형적 방법이라 오차가 적고, 방법 B는 삼각함수가 들어가서 측정한 값에 오차가 생겼을 때 값이 크게 뒤틀리는 것이 원인일 것이라고 추정한다.